

Aula 15 - Exercícios

Com base no conteúdo da aula e utilizando a famosa base de dados **Sakila** (que simula o sistema de uma locadora de DVDs com tabelas como `film`, `actor`, `customer`, `rental`, `payment`, etc.), elaborei uma lista de 20 exercícios. Eles estão divididos pelos módulos da aula para que os alunos possam praticar a evolução lógica do conteúdo.

Módulo 1: O Universo dos Joins (Junções)

1. O que aconteceria conceitualmente se executássemos a consulta `SELECT * FROM actor, film;` sem nenhuma cláusula `WHERE` ou `JOIN`? Use o termo técnico ensinado na aula para descrever esse resultado e explique por que ele geralmente não é útil.
 2. Utilizando `INNER JOIN`, escreva uma consulta que retorne o primeiro nome (`first_name`) e o último nome (`last_name`) do cliente (`customer`), junto com o endereço completo (`address`) da tabela de endereços (`address`).
 3. Precisamos listar todos os filmes da tabela `film` e, se possuírem, a qual categoria eles pertencem. Sabendo que a ligação é feita pela tabela `film_category`, utilize `LEFT JOIN` para garantir que um filme apareça no resultado mesmo que (hipoteticamente) não tenha sido associado a uma categoria.
 4. As tabelas `customer` e `payment` possuem uma coluna em comum com o mesmo exato nome: `customer_id`. Reescreva a junção entre essas duas tabelas utilizando a sintaxe simplificada `USING`.
 5. Levando em conta a base Sakila, por que utilizar um `NATURAL JOIN` entre as tabelas `customer` e `staff` pode ser uma "armadilha"? (Dica: Lembre-se do que a aula diz sobre colunas com nomes idênticos que não são chaves primárias/estrangeiras, como `last_update` ou `store_id`).
-

Módulo 2: O Desafio dos Valores Nulos (NULL)

6. Na tabela `rental`, a coluna `return_date` armazena a data em que o DVD foi devolvido. Escreva uma consulta para encontrar todos os aluguéis que **ainda não foram devolvidos** (ou seja, estão com a data de retorno em branco/nula).
 7. Agora, escreva uma consulta para listar apenas os IDs de aluguel (`rental_id`) dos filmes que **já foram devolvidos**.
 8. Se você tentar filtrar os aluguéis pendentes usando a expressão `WHERE return_date = NULL`, o banco de dados não retornará nenhuma linha. Explique, com base na lógica ternária do SQL, por que o sinal de igual (=) não funciona nesse caso.
 9. Vamos cruzar todos os filmes da tabela `film` com o inventário da loja (`inventory`) usando um `LEFT JOIN`. Adicione um filtro na cláusula `WHERE` para descobrir quais filmes do catálogo **não possuem nenhuma cópia física** na loja.
 10. Imagine que temos 50 registros na tabela `rental` onde a `return_date` é `NULL`. Se executarmos a consulta `SELECT DISTINCT return_date FROM rental WHERE return_date IS NULL;`, quantas linhas o banco de dados retornará? Justifique.
-

Módulo 3: Resumindo com Funções de Agregação

11. Utilizando a função de agregação adequada, escreva uma consulta para descobrir a quantidade total de filmes registrados na tabela `film`.
 12. O gerente quer saber qual foi a receita total da locadora. Escreva uma consulta que some a coluna `amount` da tabela `payment`.
 13. Qual é a duração média (coluna `length`) de todos os filmes do catálogo? Retorne esse número em uma única coluna chamada `duracao_media`.
 14. Utilizando apenas uma consulta e duas funções de agregação diferentes, descubra qual é o valor mais barato e o valor mais caro do aluguel (`rental_rate`) cadastrado na tabela `film`.
 15. Precisamos saber quantos clientes únicos (pessoas diferentes) realizaram pelo menos um pagamento na locadora. Escreva uma consulta na tabela `payment` combinando as funções `COUNT` e `DISTINCT` na coluna `customer_id`.
-

Módulo 4: Agrupamentos e Filtros (GROUP BY e HAVING)

16. Queremos saber quanto cada cliente gastou individualmente na locadora. Escreva uma consulta que exiba o `customer_id` e a soma do `amount` da tabela `payment`, agrupando o resultado para cada cliente.
 17. Com base na **Regra de Ouro do Rigor Técnico** ensinada na aula, explique por que a consulta a seguir gera um erro no banco de dados:

```
SELECT customer_id, staff_id, SUM(amount) FROM payment GROUP BY customer_id;
```
 18. Pegando a consulta desenvolvida no exercício 16, adicione a cláusula `HAVING` para exibir **apenas** os clientes que gastaram, no total, mais de 150 dólares na locadora.
 19. O `WHERE` filtra linhas isoladas, enquanto o `HAVING` filtra grupos. Escreva uma consulta que conte a quantidade de pagamentos (`payment_id`) feitos, agrupados por `staff_id`. No entanto, utilize a cláusula `WHERE` para contar apenas os pagamentos com valor (`amount`) superior a 2.00.
-

Módulo 5: Ordem Lógica e Execução (Unindo os Mundos)

20. O chef de cozinha (nosso banco de dados) não lê a consulta a partir do `SELECT`. Dada a consulta abaixo, descreva o "passo a passo" numérico explicando o que o motor de banco de dados faz em cada uma das 5 etapas da ordem lógica de processamento (`FROM`/`JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`).

SQL

```
SELECT c.name AS categoria, COUNT(fc.film_id) AS total_filmes
FROM category c
INNER JOIN film_category fc ON c.category_id = fc.category_id
WHERE c.name NOT IN ('Action', 'Horror')
GROUP BY c.name
HAVING COUNT(fc.film_id) > 50;
```

Exemplo de resposta esperada(início):

- *Passo 1 (FROM/JOIN):* O banco de dados vai primeiro nas tabelas `category` e `film_category` e cria um mesclado cruzando... (continue a explicação para as demais cláusulas).